> في كل مرة كنت أشوف `ToListAsync()` ، كنت مقتنع إني كده بحسن الأداء, لكن للاسف طلع مش حقيقي

في الفترة دي أنا بدأت أركز أكتر في C# وأفهمها بعمق، مش مجرد أعرف أستخدم الـ Features وخلاص.

ومن أول المواضيع اللي شدّتني كان `async` و `await` .

كنت فاكر إني فاهمهم.

أكتب `async` ، أحط `await` ، والدنيا شغالة.

لكن كل ما كنت بقرأ أكتر، اكتشفت إن فيه فرق كبير بين إن الكود **يشتغل**، وإنك **تبقى فاهم هو بيشتغل إزاي** .

وأول حاجة اكتشفت إنها كانت غلط عندي هي الفكرة دي:

> **"Async بيخلي الكود أسرع."**

وأعتقد إن ناس كتير لسه فاكرة كده.

أنا كنت واحد منهم.

كنت فاكر إن...

لو عندي API بترد في حوالي 200ms.

وحولت كل الـ Database Calls من:

```
_context.Users.ToList();
```

إلى:

```
await _context.Users.ToListAsync();
```

فالـ Response Time هيقل.

يمكن يبقى 150ms.

أو حتى 100ms.

وده كان منطقي بالنسبالي وقتها.

لأن Async = أسرع.

صح؟

...

طلع لأ.

طب استنى...

لو Async مش بيخلي الكود أسرع...

أمال Microsoft عملته ليه؟

السؤال ده هو اللي خلاني أبدأ أقرأ في الـ Documentation الرسمية، وأشوف مقالات مهندسي .NET في Microsoft، وتحديدًا حاجات كتبها Stephen Toub نفسه.

وساعتها بدأت أفهم إن المشكلة اللي Async معمول يحلها مختلفة تمامًا عن اللي كنت متخيله.

المشكلة الحقيقية

خلينا نتخيل إن عندنا API بسيطة.

```
public IActionResult GetUsers()
{
    var users = _context.Users.ToList();
    return Ok(users);
}
```

الكود ده هيبعت Query للـ SQL Server.

بعدها هيستنى لحد ما الـ Database ترد.

وأثناء الوقت ده...

أنا كنت متخيل إن السيرفر شغال.

لكن الحقيقة إن الـ Thread اللي بيخدم الـ Request بيبقى واقف مستني.

بس.

مش بيعمل أي حاجة.

وده اسمه **Blocking**.

ولما بدأت أفهم النقطة دي، بدأت الصورة تتغير عندي.

> **Note**
> في .NET، كل Request بييجي على الـ Server بياخد Thread من الـ **Thread Pool** عشان ينفذ الكود بتاعه. الـ Thread Pool ده مش عدد لانهائي — لو كل الـ Threads اتشغلوا واقفين مستنيين، السيرفر مش هيقدر يستقبل Requests جديدة، حتى لو الـ CPU نفسه فاضي.

مثال خلاني أفهم الفكرة

وأنا بقرأ، لقيت ناس كتير بتشرح Async بمثال المطعم.

وبصراحة هو من أكتر الأمثلة اللي خلت الفكرة تركب في دماغي.

تخيل إن فيه ويتر في مطعم.

في السيناريو الأول...

الويتر خد الأوردر.

وبدل ما يروح يخدم باقي الترابيزات...

فضل واقف قدام المطبخ مستني الأكل يخلص.

الأكل هيخلص في 20 دقيقة سواء وقف أو مشي.

لكن خلال الـ 20 دقيقة دول...

الويتر معملش أي حاجة.

وده شبه اللي بيحصل في الـ Sync Code.

أما في السيناريو التاني...

الويتر خد الأوردر.

وساب المطبخ.

وراح يخدم ترابيزات تانية.

ولما الأكل جهز...

رجع أخده.

الأكل برضه مخلصش أسرع.

لكن المطعم قدر يخدم عدد أكبر من الناس في نفس الوقت.

وساعتها بدأت أفهم إن Async مش بيحاول يسرع العملية نفسها...

هو بيحاول يستغل وقت الانتظار بشكل أحسن.

طب Async بيحل المشكلة دي إزاي؟

بعد ما فهمت يعني إيه Blocking...

بدأت أسأل نفسي سؤال تاني.

> طيب... لو الـ Thread واقف مستني الـ Database، إزاي Async بيحل المشكلة دي؟

في الأول كنت فاكر إن أول ما أكتب `await`، .NET بيعمل Thread جديد مخصوص للعملية دي.

وده بالمناسبة مفهوم منتشر جدًا، وغلط تمامًا.

لما بدأت أقرأ أكتر، اكتشفت إن الموضوع مش بس مختلف... هو عكس اللي كنت متخيله. الـ Async مش بيستخدم Thread إضافي، هو بيقلل عدد الـ Threads المحجوزة.

خلينا نمشي خطوة خطوة.

لو استخدمت Sync

لما الـ Request تدخل الـ API...

الـ Thread يبدأ ينفذ الكود.

ولما يوصل للسطر ده:

```
var users = _context.Users.ToList();
```

هيبعت Query للـ SQL Server.

وبعدين...

هيقف.

ويفضل مستني.

لحد ما الـ Database ترد.

طول الفترة دي...

الـ Thread محجوز.

مش بيخدم أي Request تاني.

طيب لو استخدمت Async؟

لما أكتب:

```
var users = await _context.Users.ToListAsync();
```

اللي بيحصل مختلف تمامًا.

أول ما الـ Query تتبعت للـ Database...

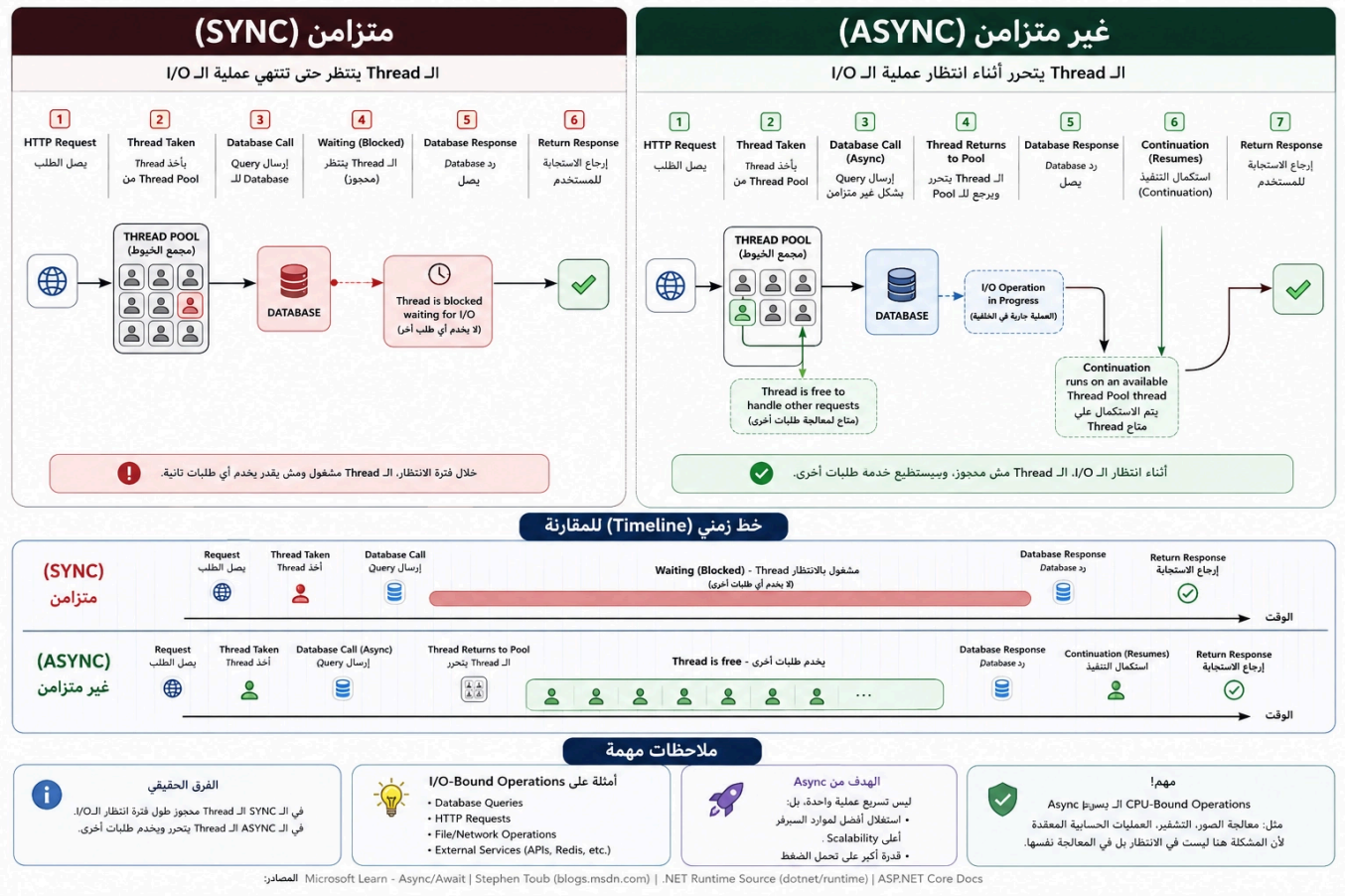
نظام التشغيل بياخد بالّه من عملية الـ I/O دي (عن طريق آلية اسمها I/O Completion Ports)، ومفيش أي Thread محتاج يفضل واقف يستنى الرد.

الـ Thread اللي كان بينفذ الـ Request **بيرجع فورًا للـ Thread Pool**، عشان يقدر يخدم Request تانية في نفس اللحظة.

ولما الـ Database تخلص وترد...

الـ Runtime بياخد Thread متاح من الـ Thread Pool (مش شرط يكون نفس الـ Thread اللي بدأ الطلب) عشان يكمل تنفيذ الكود اللي بعد الـ `await` — والجزء ده اسمه **Continuation**.

أول مرة فهمت النقطة دي، بدأت أفهم ليه الناس بتقول إن Async بيحسن Scalability.



أهم حاجة غيرت طريقة تفكيري

أنا كنت فاكر إن المشكلة هي إن الـ Database بطيئة.

لكن اكتشفت إن المشكلة الحقيقية ساعات بتكون إن الـ Threads بتفضل واقفة مستنية من غير داعي.

يعني...

الـ Database هتاخد نفس الوقت بالظبط، سواء استخدمت Sync أو Async.

لكن السيرفر هيقدر يخدم عدد أكبر من المستخدمين في نفس الوقت، لأنه مش مضيع Threads في الانتظار.

وده فرق كبير جدًا.

طب ليه الناس بتقول إن Async أسرع؟

بصراحة...

أنا أعتقد إن سبب اللخبطة إن فيه فرق بين كلمتين:

- **Faster** (أسرع لكل طلب لوحده)
- **More Scalable** (قادر يستحمل عدد طلبات أكبر في نفس الوقت)

وأنا كنت بخلط بينهم.

لو عندك User واحد بيستخدم الـ API...

غالبًا مش هتحس بفرق كبير، وممكن حتى تحس إن الـ Async أبطأ شوية زي ما هنشوف بعدين.

لكن تخيل إن عندك 1000 مستخدم بيضربوا نفس الـ Endpoint في نفس الوقت.

هنا Async هيبدأ يبان تأثيره الحقيقي.

مش لأنه خلّى كل Request تخلص أسرع...

لكن لأنه سمح للسيرفر يتعامل مع Requests أكتر من غير ما يحتاج Threads أكتر.

ودي نقطة حسيت إنها مهمة جدًا، لأنها غيرت تعريف كلمة "Performance" عندي.

الأداء مش معناه دائمًا إن العملية تخلص في وقت أقل.

ساعات الأداء الأفضل معناه إن التطبيق يقدر يتحمل ضغط أكبر بنفس الموارد.

> **Note**
> ده بالظبط سبب إن Microsoft بتنصح باستخدام async في ASP.NET Core حتى لو مش حاسس بفرق في الأداء وانت بتعمل تست على جهازك لوحدك. الفايدة الحقيقية بتظهر تحت ضغط (Load)، مش على طلب واحد.

طب ليه الـ Response Time ممكن يزيد؟

دي كانت أكتر نقطة استغربتها وأنا بقرأ.

لأن لو Async معمول علشان يحسن استخدام الـ Threads...

ليه ساعات بنلاقي الـ Response Time زاد شوية؟

بعد ما قرأت في Documentation الخاصة بـ Microsoft، اكتشفت إن استخدام `async` مش "ببلاش".

فيه تكلفة، لكنها غالبًا ضئيلة جدًا مقارنة بزمن عمليات الـ I/O زي الـ Database أو الـ HTTP Calls.

يعني لما تكتب:

```
public async Task<User> GetUserAsync()
{
    return await _repository.GetUserAsync();
}
```

الـ Compiler بيحول الـ Method دي إلى **State Machine** (كائن بيحتفظ بحالة تنفيذ الـ Method) علشان يعرف:

- فين هيقف عند `await`.
- وإزاي يرجع يكمل بعد كده، وهو معاه نفس الـ Local Variables اللي كانت موجودة قبل الوقفة.

الـ State Machine ده بياخد شوية Memory Allocation إضافية، وفيه Overhead بسيط في تنفيذه مقارنة بكود متسلسل عادي.

في أغلب التطبيقات، التكلفة دي صغيرة جدًا ومش هتحس بيها، لأنها بتتقاس بأجزاء من الميلي ثانية، بينما الـ Database Call نفسه بياخد ميلي ثواني كتير.

لكن وجودها هو السبب إن Async مش دايمًا هيخلي الـ Response Time أقل — وممكن كمان يخليه أعلى بجزء بسيط جدًا لو العملية أصلًا سريعة جدًا (زي عملية في الـ Memory من غير I/O حقيقي).

وده طبيعي.

لأن الهدف الأساسي أصلًا مش تقليل زمن تنفيذ الـ Request الواحد.

الهدف هو إن السيرفر يقدر يخدم Requests أكتر بنفس الموارد.

إمتى أستخدم Async؟

وأنا بتعلم، كنت ساعات بحس إن لازم أي Method أكتبها تبقى Async.

لكن بعد شوية فهمت إن الموضوع مش كده.

لو العملية **I/O-bound** — يعني بتستنى حاجة خارجية زي:

- Database
- HTTP Request
- File System
- Cloud Storage
- Redis

يبقى Async غالبًا هو الاختيار الصح، لأن فيه وقت انتظار حقيقي ممكن تستغله.

أما لو العملية **CPU-bound** — يعني كلها شغالة على المعالج، زي:

- معالجة الصور
- ضغط الملفات
- التشفير
- عمليات حسابية معقدة

فالـ Async مش هيخليها تخلص أسرع، لأن مفيش وقت انتظار أصلًا؛ الـ CPU هو اللي شغال طول الوقت ومحتاج يخلص شغله.

> **Note**
> في الحالة دي، لو حابب تمنع العملية من إنها تحجز الـ Thread الأساسي (مثلًا في تطبيق Desktop عشان الواجهة متجمدش)، الحل مش `async/await` العادي، لكن `Task.Run()` عشان تنقل الشغل لـ Thread تاني من الـ Thread Pool. الفرق إن هنا فعلاً بتستخدم Thread إضافي لتوزيع الحمل، مش لتقليل الانتظار.

حاجة كنت بعملها... وطلعت مش صح

وأنا بتعلم، قابلت الكود ده في أكتر من مكان.

```
// ❌ Anti-pattern
await Task.Run(() => _context.Users.ToListAsync());
```

في الأول افتكرت إن دي طريقة تخلي الـ Database أسرع.

لكن بعد ما فهمت الموضوع أكتر، اكتشفت إن ده مش بس مبيحققش مكسب... هو فعليًا بيضر الأداء تحت الضغط.

ليه؟

لأن عملية قراءة البيانات من الـ Database أصلًا عملية I/O، مش CPU.

يعني اللي إحنا مستنيينه هو رد الـ Database، مش إن الـ CPU يخلص شغل.

فلما أحطها جوه `Task.Run()`:

1. أنا باخد Thread من الـ Thread Pool عشان يبدأ العملية.
2. الـ Thread ده هيدخل جوه العملية الـ Async أصلًا، وهيرجع للـ Thread Pool تاني بسرعة (لأنها I/O).
3. النتيجة: استخدمت Thread إضافي من غير أي داعي حقيقي، وزودت ضغط على الـ Thread Pool من غير فايدة.

الشكل الصح والمباشر هو:

```
// ✅ Correct
var users = await _context.Users.ToListAsync();
```

ودي من الحاجات اللي ناوي أقرأ فيها أكتر، لأنها تستحق موضوع كامل لوحدها (خصوصًا الفرق بينها وبين استخدام `Task.Run` في تطبيقات الـ Desktop أو الـ Console، اللي فيه أحيانًا بيكون له داعي).

الخلاصة

لو فيه حاجة واحدة غيرت طريقة تفكيري بعد ما قرأت عن Async، فهي دي:

> الـ Async مش معمول علشان يسرع الكود.

الـ Async يسمح للـ Thread إنه يتحرر أثناء انتظار عمليات الـ I/O، بدل ما يفضل واقف من غير فايدة.

وده معناه:

✅ استغلال أفضل لموارد السيرفر.

✅ قدرة أعلى على تحمل الضغط (Scalability).

✅ عدد أكبر من الـ Requests المتزامنة بنفس عدد الـ Threads.

لكن...

مش شرط إن كل Request تخلص أسرع. وممكن حتى تلاقي فرق بسيط جدًا في الاتجاه العكسي بسبب تكلفة الـ State Machine.

وده كان أكبر فرق بين اللي كنت فاكره، واللي فهمته بعد ما رجعت للمصادر الرسمية.

> 📊 **Diagram Idea**
> جدول بسيط بعمودين: "Sync" و "Async"، وصفوف: عدد الـ Threads المحجوزة تحت ضغط، الـ Response Time لطلب واحد، القدرة على استيعاب Load عالي. ده هيوضح بصريًا إن الفرق مش في السرعة، لكن في القدرة على التحمل.

إنت كنت فاكر إيه؟

أنا عن نفسي كنت فاكر إن:

```
Async = Faster.
```

لكن واضح إن الصورة أكبر من كده بكتير.

لو كنت فاكر نفس الحاجة، يبقى إحنا اتعلمنا درس جديد النهارده. 😄

المصادر

المقال ده مبني على قراءة وتلخيص من المصادر الرسمية التالية:

- Microsoft Learn — Asynchronous programming with async and await
- Microsoft Learn — ASP.NET Core Performance Best Practices

- Stephen Toub (Principal Engineer, .NET Team) — مقالات عن Async internals والـ Thread Pool
- .NET Runtime Documentation
- GitHub — dotnet/runtime
- GitHub — dotnet/aspnetcore

لو لقيت أي معلومة محتاجة تصحيح أو توضيح، ابعتلي.

أنا بكتب السلسلة دي علشان أوثق رحلتي في تعلم .NET، وأتمنى إنها تفيد أي حد ماشي في نفس الطريق.